



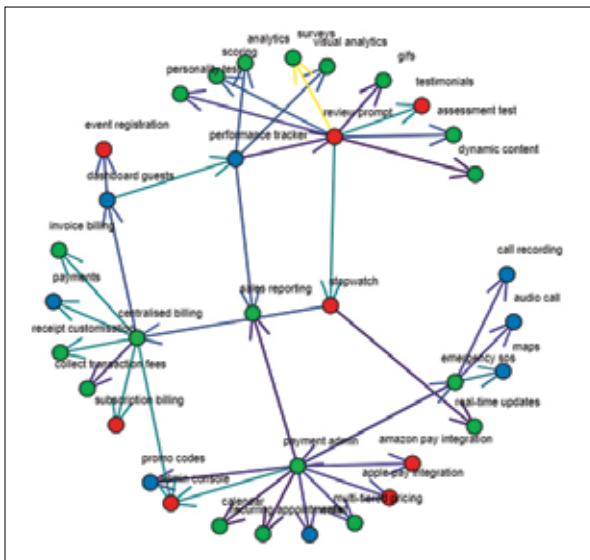
Entire CS curriculum

A 1000 Youtube video collection/archive, covering every skill essential for a Computer Science engineer. <http://dgit.in/feb21-83>



Colliding Galaxies ASCII

Try out this fun little ASCII simulation of Colliding Galaxies in C++, using Newton's law of universal gravitation. <http://dgit.in/feb21-84>



A knowledge graph

Follow-up: How do you utilise AI during the app development work-flow?

SDD: Builder.ai has deepened its platform and we are proud to share that many parts of the core platform within the Builder is powered by Artificial Intelligence (AI) and Machine Learning (ML), with different degrees of evolution. Many of our AI use cases are powered by our proprietary Builder Knowledge Graph, our AI brain. The key applications include feature recommendations and personalisation, custom timeline, pricing estimation, automated clickable prototype creation, creator resource allocation and partly autonomous project delivery/management.

Furthermore, we have leveraged and implemented recent advances in natural language processing and computer vision within the platform. Applications of the former include: handling custom features, mapping problem utterances to templates, named entity recognition in spec calls and tracker story curation.

Computer vision techniques power our creator identity detection as well as one of our flagships subsystems: VisualQA that reduces the time to detect defect detection of UI from weeks to seconds using a deep learning system.

Some of these applications will continue to evolve over time. As we

gather more data from our ecosystem, our AI capabilities will continue to become more powerful. For instance, our recommendations will become more contextual and VisualQA will become an expert at identifying defects.

Considering that clients might have an enormously fragmented tech stack with which they would want to build their native apps,

how does Builder.ai service such a fragmented tech base?

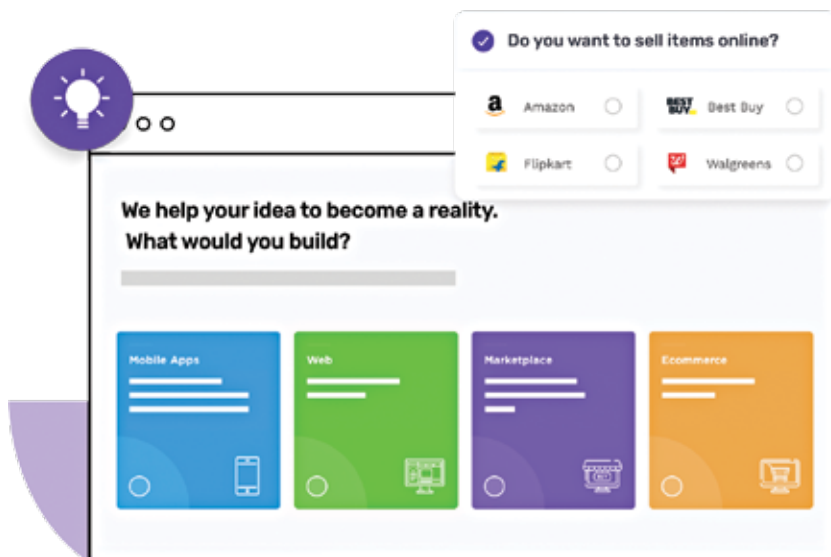
SDD: First and foremost we don't piece-meal build software; our applications are built as full systems and we have full support for any API that a customer may have. That being said; the tech stack for our delivery is governed by our platform and not customer dependent – this allows us to leverage our reusable features.

Our product delivery mechanism ensures that the customer's end-requirements are fulfilled regardless of

the tech stack at the customer's end.

Right from the prototype generation using Builder Now, our customers can visualize their application by generating a clickable prototype. With the help of Builder Studio, customers can add on many features which are our reusable building blocks that can be injected into any project for assembly. Around 500 features make up 80% of all software (features that you use everyday, like login, secure payments or shopping cart) and we've coded those already. We call them building blocks and you can fit them together however you like.

Furthermore, all these blocks communicate via the RunEngine to ensure seamless integration. Our User Interface (UI) Engine can generate React & React Native code for a given Sketch file, which is human-readable, easily extensible, and responsive across multiple platforms and screen resolutions. The design templates that we provide act as starting points for our designers to customize the designs as per the customer's requirements. As an added measure to ensure accuracy, the delivery team receives a set of questions to ask the customer to guide the specification process, as well as templated user stories for the customisation of the project.



Builder Studio